

MODULE 1 – SELENIUM PYTHON

Assignment Report

Student	Ishani Mandal
Program	B.Tech – Computer Science and Information Technology
Module	Module 1
Technology	Python and Selenium WebDriver
Browser	Google Chrome

1. Overview

This report is an automation effort we completed in Module 1 with Python and Selenium WebDriver to test the Selenium product. The assignments emphasize automation of the web, explicit waits, dealing with checkboxes and auto-completes, JavaScript alerts, handling tables, multiple browser windows and iframe interactions.

Create an automation for logging in to a Selenium website.

2. Assignment 1 – Selenium Login Automation

Problem: Trying to automate the login procedure of the SauceDemo website using Python.

Task: Automation of the website SauceDemo using Python and Selenium.

Objective: Open SauceDemo, enter the user detail, click on Login button and confirm that the user is logged in successfully.

Implementation: Selenium WebDriver with ID, Name and XPath Locators. The username **standard_user** and the password **secret_sauce** was typed and the Login button was pressed.

Verification: The following URL was used to verify the presence of the file **/inventory.html**.

Result: Login went fine and the current URL was printed.

Outcome: Identify and interact with the basic Selenium element using locators.

3. Assignment 2 – Explicit Wait

Goal: Dynamically load an element and check the contents of the element after load.

The Dynamic Loading page was displayed and the Start button was hit. WebDriverWait with the wait for the result element was performed using the `expected_conditions.visibility_of_element_located()` condition.

Verification: The message that is displayed is verified against **Hello World!**.

Result: The message to be expected was shown and the assignment was passed.

Key concept: An explicit wait allows Selenium to wait for an element condition while a fixed delay is not enough.

4. Assignment 3 – Checkbox and Auto-complete Handling

Select all the checkboxes and auto-complete fields, then click Submit.

Goal: Use Selenium to select a checkbox and to take action on one of the auto-completing suggestions.

Implementation: All checkbox elements were found on the Checkboxes page and the first checkbox was checked if it was not already picked. The DemoQA Auto Complete page was then opened. The value **Red** was entered and all the suggested ideas were noted and printed.

Verification: The first checkbox was checked off and the suggestion **Red** was selected.

Result: Successfully performed checkbox selection and auto-complete selection.

Key concept: To handle multiple elements and dynamic suggestions via Selenium waits and element states.

5. Assignment 4 – JavaScript Alerts

Objective: Respond to JavaScript pop-up alerts, confirm and prompt and confirm their results.

Implementation: We used driver.switch_to.alert method to switch to the alert in the browser using Selenium. The Alert was accepted, the Confirm is dismissed and text was placed in the Prompt before accepting it.

Verification: The alert message was checked, confirming successful click, the confirm message showed **You clicked: Cancel**, and the following prompt appeared: **You entered: Ishani**.

Result: All three types of JavaScript pop-ups were dealt with appropriately.

Key concept: Switching to browser alerts and using accept(), dismiss() and send_keys().

6. Assignment 5 – Web Table Handling

Objectives: Read data from a web table, and locate the record that has the last name Smith.

Implementation: The table table1 was found. The web page table headers and body rows were extracted using Selenium. The cells in each row were turned into a list of cell values and then compared to the search name.

The last name was correct, the Last Name, First Name, Email, Due, Web Site and Action values were printed.

Result: The Smith record was successfully found and assignment passed.

Key concept: Use XPath to traverse rows and cells in tables and use find_elements() to find cells.

7. Assignment 6 – Multiple Windows and iFrame Handling

Check out the following resources to understand how to navigate multiple windows and iFrame.

Objective: Switching Browser Windows and interacting with an iframe with Selenium.

Implementation: Save original window handle before opening new window. Selenium waited until two window handles were available, and switched to the new window, where the heading **New Window** was checked. The driver then restored the original window.

An iFrame page was opened and Selenium waited for the iFrame to become available. The driver entered into the iframe, located the TinyMCE editor and inserted the text **Selenium Iframe Test** using JavaScript. The driver was then switched back to the default content.

Verification: Assertions were used to verify the new-window heading and iframe text.

Result: Operations on Multiple Windows and Multiple Iframes are successful.

Key concept: Switching between windows, iframe context switching and return to default content.

8. Environment and Dependencies

- Python
- Selenium WebDriver
- Google Chrome

9. Learning Outcomes

- Developed simple automation scripts for browsers using Selenium with Python.
- Used various locators of Selenium – ID, Name, XPath and Link Text.
- Made use of explicit waits and expected conditions for dynamically loaded elements.
- Managed check boxes and auto completion suggestions.
- Designed and used JavaScript Alert, Confirm and Prompt boxes.
- Read and searched data from web tables.
- Alternated between various browser windows and was using IFRAME.
- Tested for expected outcome using assertions during automation.

Summary: In Module 1 we got hands-on practice using the basic and commonly used features of Selenium WebDriver. The assignments were geared towards various real-life scenarios of web automation and provided a basis for future assignments.